Module 6 – Hands-on Sessions

CS5013 – Programming with AI
Building Software with LLM APIs

V. Krishna Nandivada
Department of Computer Science and Engineering, IIT Madras

Overview

Module 6 has two required hands-on sessions plus an optional Skill-authoring exercise:

- **Session 6A – Retrieval-augmented Q&A over a local Java repository.** You index the repo, retrieve top-k chunks for a natural-language query, and ask the LLM to answer using only those chunks.
- **Session 6B – Agentic mini-pipeline with tool calls.** You expose two Java tools, wire them to an LLM, and drive a short ReAct-style loop with a budget and a trace log.

Session 6A uses the starter under `handson/m6-lab-a/`; Session 6B uses `handson/m6-lab-b/`. Both are self-contained copies of the shared `handson/m6/` scaffold – you can work on either without touching the other.

Setup

- Download the starter for each session you will attempt:
  - Session 6A – https://www.cse.iitm.ac.in/~krishna/cs5013/m6-a-pub.tgz
  - Session 6B – https://www.cse.iitm.ac.in/~krishna/cs5013/m6-b-pub.tgz

  Unpack each to `handson/m6-lab-a/` and `handson/m6-lab-b/` respectively.
- Java 17+ (21 recommended) and `make` on the PATH.
- `curl`, `unzip`, and `tar` (for the `deps` target).
- **Only if you want to swap in a real LLM**: Apache Maven on the PATH plus Ollama (see below). The default stubs run entirely offline with no model at all, so you can start either lab, finish the TODOs, and run the full test suite before you ever install anything beyond a JDK.

The Setup, langchain4j, and Ollama steps that follow apply identically to both labs.

Installing langchain4j (one-time)

The `Makefile` ships a `langchain4j` target that pulls langchain4j and its transitive dependencies into `libs/langchain4j/` using Maven:

```
cd handson/m6-lab-a # or m6-lab-b -- run once per lab
make langchain4j # one-shot; requires Maven on PATH
make build # subsequent builds pick the new jars up
```

Installing and using Ollama (offline LLM)

Ollama runs an LLM entirely on your laptop or desktop. No account, no key, no network calls once the model is pulled.

Install.

- *macOS.* `brew install ollama` (or download the `.dmg` from https://ollama.com/download).
- *Linux.* `curl -fsSL https://ollama.com/install.sh | sh`.
- *Windows.* Download the installer from https://ollama.com/download/windows.

Start the daemon. On macOS/Linux the installer registers a background service. If it is not already running:

```
ollama serve & # or: brew services start ollama
```

The daemon listens on `http://localhost:11434` by default.

Pull a model. Pick one that fits your RAM; the labs do not need a frontier model. Rough guide:

- 8 GB RAM machine: `llama3.2:3b` (~2 GB) or `qwen2.5:3b`.
- 16 GB RAM: `llama3.1:8b` (~5 GB) or `mistral:7b`.
- 32 GB RAM: `llama3.1:70b` (~40 GB; only if you have patience and SSD space).

```
ollama pull llama3.1:8b # one-shot download
ollama list # confirm it is installed
```

Smoke-test from the shell. Before touching the Java code, confirm the daemon answers:

```
curl http://localhost:11434/api/generate -s \
  -d '{"model":"llama3.1:8b","prompt":"Say hi.","stream":false}'
```

You should get a JSON reply with a `response` field.

Wire into the lab. The lab has a reference implementation at `example/OllamaLlm.java`.

```
import dev.langchain4j.model.ollama.OllamaChatModel;
import dev.langchain4j.model.chat.ChatLanguageModel;

public class OllamaLlm implements LlmClient {
    private final ChatLanguageModel model = OllamaChatModel.builder()
            .baseUrl("http://localhost:11434")
            .modelName("llama3.1:8b")
            .temperature(0.0)
            .build();

    @Override
    public String complete(String prompt) {
        return model.generate(prompt);
    }
}
```

After `make langchain4j`, copy the file into `src/` (`cp example/OllamaLlm.java src/`) and change every construction site that currently says `new StubLlmClient()` to `new OllamaLlm()`.

Where exactly are those construction sites? `RagAssistant` (lab-a) and `AgentDriver` (lab-b) never instantiate `StubLlmClient`; they only hold an `LlmClient` reference, so those classes do not need any edit. `new StubLlmClient()` appears in two kinds of place across the two labs:

- **m6-lab-a** – in `src/Main.java`, in the two lines of the "Pipeline construction" block:

```
Embedder emb = new StubEmbedder(); // -> MiniLmEmbedder
LlmClient llm = new StubLlmClient(); // -> new OllamaLlm()
```

  `RepoIndexerTest` does not touch `RagAssistant`, so there is nothing to change in the test tree until you add your own test.

- **m6-lab-b** – in `test/AgentDriverTest.java`, which already constructs the agent as `new AgentDriver(new StubLlmClient(), ...)`. Lab-b does not ship a `Main`; when you write one to drive the agent interactively, that is the second site to swap.

Swap `new StubLlmClient()` for `new OllamaLlm()` at those sites (and optionally the embedder for a real `MiniLmEmbedder` in lab-a) once `make langchain4j` has installed the jars.

Common pitfall

A local 7B/8B model is *slow* compared to a hosted frontier model: expect 5–30 seconds per RAG answer on a laptop. Keep your prompts short and your top-k small while iterating.

Starter contents

`m6-lab-a/` (Session 6A – RAG):

- `sample-repo/` – a small Java library-management project (books, members, loans, fines; ∼20 classes). The corpus you index.
- `src/RepoIndexer.java` – stub chunker + embedder (the real TODOs: `chunkFile` and `embed`).
- `src/RagAssistant.java` – stub retrieve + prompt-assembler (the real TODOs: `retrieve` and `buildPrompt`; `answer` is pre-wired).
- `src/{Chunk,Embedder,LlmClient,StubEmbedder,StubLlmClient}.java` – data record, interfaces, stubs.
- `src/Main.java` – end-to-end driver (invoked by `make run`): builds the pipeline, indexes `sample-repo/`, and loops over a `String[] QUESTIONS` array printing retriever hits + LLM answer per question. Ships with one seed question; task (5) asks you to add nine more.
- `test/RepoIndexerTest.java` – seed tests.
- `prompts/rag.txt` – the RAG generation prompt template with two placeholders: `{query}` (the student's question) and `{context}` (the serialised top-k chunks). Loaded at run time by `RagAssistant.buildPrompt(...)`; a plain text file so you can edit the instruction (e.g. tighten the anti-hallucination guard) without recompiling.

`m6-lab-b/` (Session 6B – ReAct agent):

- `books.json` – catalogue the agent reasons over.
- `src/AgentDriver.java` – stub ReAct loop (the real TODOs: `shouldStop` and `parseAction`).
- `src/tools/{ListBooksTool,SearchBooksTool}.java` – stub tool bodies (the real TODOs).

- `src/{Tool,LlmClient,StubLlmClient}.java` – interfaces and the chat stub.
- `test/AgentDriverTest.java` – seed tests (constructs the agent with `new StubLlmClient()`).
- `prompts/react.txt` – ReAct reasoning template.

Common to both labs:

- `Makefile` – `deps`, `build`, `test`, `langchain4j`, `clean`.
- `pom.xml` – used only by `make langchain4j`.
- `example/OllamaLlm.java` – reference `LlmClient` adapter; copy into `src/` once langchain4j is installed.

1 Session 6A: RAG over a local Java repo

Objectives

- Chunk a Java repository at method granularity.
- Compute embeddings for each chunk and keep them in an in-process list for cosine search.
- At query time, retrieve top-k chunks and send them to an LLM with an anti-hallucination instruction.
- Evaluate retrieval and generation separately.

Prerequisites

- Module 6 lecture attended, particularly Sections 4–5.
- Working make build; can run `make test`.
- Optional: Ollama running locally if you want to replace `StubLlmClient` with a real model (see Setup above).

Tasks

(1) Read `RepoIndexer.java`. Fill in `chunkFile(Path)` so each Java file is split at method boundaries. For each method chunk:

- Prefix the chunk text with the file path.
- Include the enclosing class signature.
- Preserve any leading JavaDoc.

A quick JavaParser (or tree-sitter) call is easiest; avoid a regex-only splitter for anything non-trivial. A brace-balanced hand-rolled splitter is also acceptable for this corpus.

(2) The embeddings themselves flow through the `Embedder` interface. The starter's `StubEmbedder` produces a deterministic 32-dim vector that is fine for exercising the pipeline. When you want a real semantic signal, construct the indexer with a real `Embedder` (see the "Switch to real models" note in Setup) – no code change inside `RepoIndexer` is needed, because `embed` delegates straight through the interface.

(3) The lab ships a small `src/Main.java` that calls `indexDirectory(Path.of("sample-repo"))` and prints the chunk count plus wall-clock time. Run `make run` *twice* and compare:

- Before you touch `chunkFile`: the stub emits one chunk per file – so chunk count equals the number of `.java` files under `sample-repo/` (approximately eight).
- After you finish task (1): chunk count rises to roughly the total *number of methods* across those files (several dozen).

Record both numbers. The ratio is a quick sanity check that your method-granularity chunker is actually splitting on methods, not accidentally merging them. (You can ignore the per-question output on this run – that's for task (5); the retriever and prompt-assembler are still stubs at this point, so the answer will be garbage.)

(4) Read `RagAssistant.java`. `answer(String query, int k)` is already written – do *not* edit it. The real TODOs are the two helpers it calls:

- `retrieve(String query, int k)` – embed the query, cosine-rank every indexed chunk against it, return the top k. The stub body returns the first k chunks unranked; replace it.
- `buildPrompt(String query, List<Chunk> hits)` – load `prompts/rag.txt`, substitute the `{query}` and `{context}` placeholders (concatenate hits as "`source:line` + *content*" blocks), return the assembled prompt. The stub returns the raw template with the placeholders intact.

(5) Ask ten questions. Open `src/Main.java` and extend the `QUESTIONS` array – the lab ships one seed question; you add nine more. The sample repo is a small library-management system (books, members, loans, fines), so your questions should target that domain. Starting points you may use or ignore:

- "What class holds the catalogue of books, and how is it indexed?"
- "How does the system decide a loan has been returned?"
- "What is the maximum fine the policy allows?"
- "Which methods write to the loan registry?"

After editing, `make run` re-runs every question and prints, per question: the top-8 chunk locations the retriever picked and the LLM's answer. For each question, record the answer, those eight chunks, and your own judgement of correctness.

(6) Compute a rough retrieval quality signal: for each query, was the "right" chunk in the top-8? Report the fraction.

(7) Break the retriever. Change the chunker to split every 50 lines instead of at method boundaries. Re-run the ten questions. Report the fraction change. This gives you a feel for why chunking dominates quality.

(8) Restore the good chunker. Add one more entry to `QUESTIONS` in `Main.java` whose answer is *not* in the corpus (e.g. about a framework the repo does not use, or an unrelated domain). Re-run `make run`. Confirm that with the anti-hallucination instruction in `prompts/rag.txt` the LLM says "I do not find that in the passages." Now delete those two instruction lines from `prompts/rag.txt`, re-run, and watch it happily invent an answer. Put the instruction back when done.

Expected deliverables (checked in at end of session)

- `RepoIndexer.java` and `RagAssistant.java` filled in.

- `Main.java` with your extended `QUESTIONS` array (and, if you did task 8, the restored `prompts/rag.txt`).
- `6A-notes.md` with:
  - Indexer stats (chunks, seconds).
  - Ten questions, ten answers, retrieval hit-rate.
  - Result of the "bad chunker" experiment.
  - Result of the "question not in corpus" experiment.
- Commit on your branch for this session.

Common pitfall

Cranking k up to compensate for a bad chunker. This is expensive (more tokens per query) and only occasionally helps. Fix the chunker instead; a good top-4 outperforms a bad top-16.

Common pitfall

Storing embeddings without recording the model name and version. If you regenerate the store with a different embedding model and query with the old one, similarity scores become nonsense. Fail loudly at query time if the versions mismatch.

2 Session 6B: Agentic mini-pipeline

Objectives

- Register two Java functions as LLM-callable tools.
- Drive a ReAct-style loop with an explicit step budget and a trace log.
- Observe how the model chooses tools, and how the choice goes wrong when the descriptions are vague.

Prerequisites

- Module 6 lecture attended, particularly Section 3.
- Optional: Ollama running locally if you want the agent loop to drive a real model instead of the stub.
- Session 6A does not need to be done first, but familiarity with the shared scaffold helps.

The scenario

The agent has to answer questions about a small *book-catalogue* JSON file, e.g., "how many books did Ursula Le Guin write in the 1970s?" The answer requires reading the file and doing a small filter. The LLM cannot see the file directly; it must call tools.

Tools the agent is given

1. `list_books()` – returns the whole catalogue. Big and expensive to include in a prompt.
2. `search_books(author, year_from, year_to)` – returns only matching books. Cheap and targeted.

The interesting design question: will the agent pick the cheap tool, or the greedy one?

Tasks

(1) Read `AgentDriver.java`. Fill in the two tool bodies (`list_books` and `search_books`) so they read `books.json` and return the appropriate results.

(2) Read `prompts/react.txt`. Note that it instructs the model to produce a `Thought` then a tool call, and to stop when it has a final answer.

(3) Run the driver with question A: *"How many books did Ursula Le Guin write in the 1970s?"* Watch the trace log. Report: which tool was called, with what arguments, and how many iterations the loop took.

(4) Deliberately "vague-ify" the tool descriptions: e.g., replace "Return only matching books" with "Search for books." Re-run. Try two other replacements of your choice. Report the changes in tool choice.

(5) Restore the sharp descriptions. Set the `stepBudget` constructor argument to 3 at your agent construction site. Ask a harder question that would need at least 3-4 tool calls (e.g. comparing two authors' publishing decades). Confirm the driver aborts cleanly at the budget.

(6) Add a cost budget (in tokens) and re-run. Confirm it fires when expected.

(7) Write a short `6B-reflection.md` covering:

- Which tool did the agent prefer, and why do you think so?
- How much did the tool-description quality affect behaviour?
- What would a real production driver need on top of what you built?

Expected deliverables

- `AgentDriver.java` filled in, including budgets.
- Trace log for each experiment saved under `traces/`.
- `6B-reflection.md`.
- Commit on your branch for this session.

Common pitfall

Believing the trace log because the model wrote it. The `Thought` step is the model narrating its own reasoning – it can be honest, it can be a rationalization, or it can be wrong. Use the trace to see *what happened*, not to trust that it was correct.

Common pitfall

No step budget. Without a hard iteration cap, a confused agent will happily call tools forever. Ship the budget before you ship the driver.

Wrap-up

By the end of these sessions you should have:

- A working RAG loop over a real Java codebase (6A).
- An intuition for the most common failure mode in retrieval-augmented systems: bad chunking.
- An agentic loop calling real Java functions (6B), with hands-on evidence of how tool-description quality drives tool choice.

Session 6A feeds directly into Assignment 11, which asks you to extend 6A to a larger corpus with retrieval-quality measurement. Session 6B feeds into Assignment 12 (extend the agent with a third tool and a mutation to observe how it adapts).